PART 1 draw

Data Flow:

> **Note**
>
> Input → Hidden → Output
> vector → matrix multiply → activation → next vector

```
Input:     [x1, x2]

Weights:   W1 (2x3)

Hidden:    h = activation(W1 * input + bias)

Weights:   W2 (3x1)

Output:    y = activation(W2 * h + bias)
```

GRAPHICS:

Concept	Visual
Neuron	Circle
Weight	Line
Activation	Color intensity
Layer	Column

```
#include <SFML/Graphics.hpp>
#include <iostream>
#include <vector>
using namespace std;
using namespace sf;
int main() {
    RenderWindow window(VideoMode(1000, 700), "Neural Network Visualizer");
    //set frame rate limit for smooth animations
    window.setFramerateLimit(60);

    //NEURONS :
    vector <CircleShape> neurons; //NOTE THIS IS VECTOR ARRAY NEURON

    //Input layer: 2 neurons

    for (int i = 0; i < 2; i++) {
        CircleShape neuron(20);             //THIS JUST NEURON OBJECT
        neuron.setFillColor(Color::Red);
        neuron.setPosition(100, 200 + i * 100);
        neurons.push_back(neuron);          //THE NEURON OBJECT IS PUSHED INTO THE VECTOR ARRAY NEURON
    }

    //hidden layer: 3 neurons
    for (int i = 0; i < 3; i++) {
        CircleShape neuron(20);
        neuron.setFillColor(Color::Blue);
        neuron.setPosition(400, 150 + i * 300);
        neurons.push_back (neuron);

    }
    //output later: 1 neuron
    CircleShape OutputNeuron(20);
    OutputNeuron.setFillColor(Color::Red);
    OutputNeuron.setPosition(700, 300);
    neurons.push_back(OutputNeuron);

    vector < CircleShape> neuron(20);

    //main game loop:

    while (window.isOpen())
    {
        Event event;
```

```
        while (window.pollEvent(event)) {
            if (event.type == event.Closed)
                window.close();
            }

        window.clear(Color::Black);

        for (auto& neuron : neurons) {
            window.draw(neuron);
        }

        window.display();
    }

}
```

Vector is a dynamic array that ca grow when needed/
Take new element [x] push to the end of the array

```
vector <int> v;
v.push_back(5);

Output:
[5]

vector <int> v;
v.push_back(10)

output:
[5,10]
```

Mechanism

Vector has capacity (allocated memory).
When full, it allocates bigger memory.
Copies old elements → adds new ones.

Old: [5,10,] → *capacity = 2*
.push_back(20)

[5,10,20,,] → *expanded capacity*
copied: [5,10]

added:20

New array everytime?

Does it create a new array every time .push_back is called?

> It **does NOT create a new array every time** it only reallocates and copies when capacity is full; otherwise it just adds the element in-place.
>
> When capacity is full, the vector allocates a new larger **contiguous** block, copies elements there, and frees the old block, because it cannot extend in place (not necessarily due to fragmentation, but because the next memory isn't guaranteed free).

- Vector **must stay contiguous**
- If it can't grow in-place → **relocate entirely**

// DRAW

```
//draw
for(auto &neuron :: neurons){
window.draw(neurons)
}
```

Auto: Asks compiler to figure out type directly.
&: Pointer referring to neurons so the change takes place directly in real objects
Without pointer: Compiler would create copies which is slower.